Backpropagation Through Time: What It Does and How to Do It

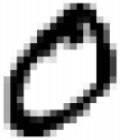
Paul Werbos

Presenter: Po-Yao (Bernie) Huang
9/16/2015

Outline

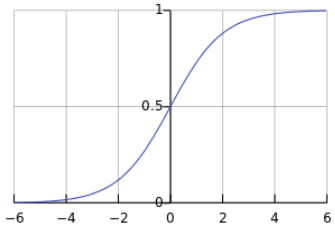
- Back Propagation
  - Recap: Supervised Learning
  - Topology
  - Learning Rule
  - Implementation details
- Back Propagation Trough Time
  - How it works
  - Implementation details
- Conclusion

Recap: Supervised Learning

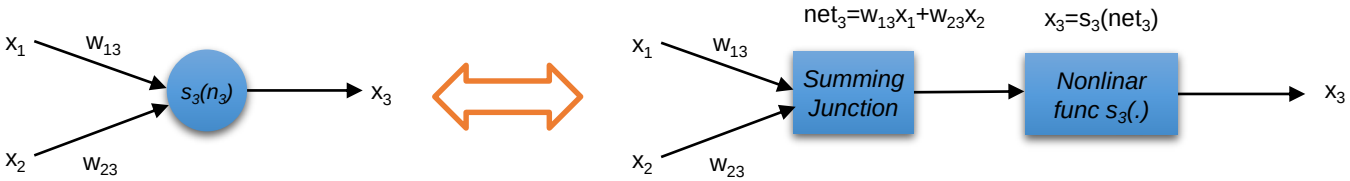
- Example Task: Hand-written digit recognition  → 0(0000)
- Dataset (Training)
  - Size: 2000 example inputs of hand-written digits: $\mathrm{T} = 2000$
  - Feature: Each input is a 19x20 pixel image
    - $\boldsymbol{X}(t) = [X_1(t), X_2(t), \dots, X_m(t)], m = 380, 1 \leq t \leq T$
  - Label: a digit 0~9 (4 bits)
    - $\boldsymbol{Y}(t) = [Y_1(t), Y_2(t) \dots, Y_n(t)], n = 4,\ 1 \leq t \leq T$
- Artificial Neural Network
  - The network with topology $\mathbf{S}$ and parameters $\mathbf{W}$ ouputs an estimate of $\widehat{\boldsymbol{Y}}(t)$ based on $\boldsymbol{X}(t)$
  - $\widehat{\boldsymbol{Y}}(t) = N(\boldsymbol{X}(t), \mathbf{W}, \mathbf{S})$

Recap: Supervised Learning

- Need to specify:
  - Network topology
    - How the network is connected
    - The perceptron functions



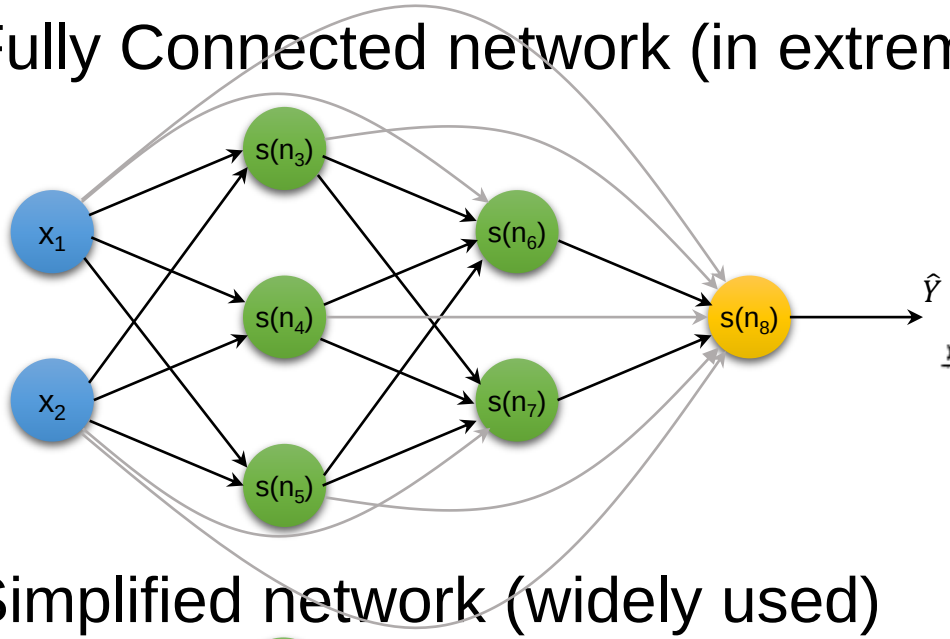
$$s(z) = \frac{1}{1+e^{-z}}$$
$$s'(z) = s(z)(1-s(z))$$

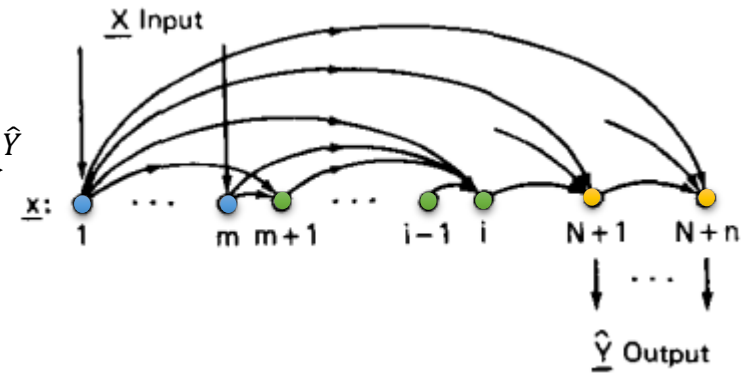


  - Learning rule:
    - The objective/error function
    - How to find the best weights $\mathbf{W}$ that $\widehat{\boldsymbol{Y}}(t)$ approximates $\boldsymbol{Y}(t)$
      - **Back Propogation !!**

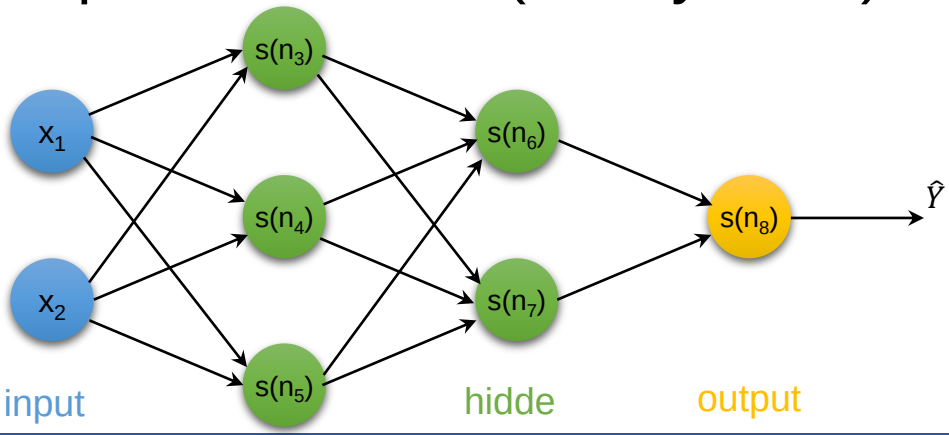
Network Topology

- Fully Connected network (in extreme)





- Simplified network (widely used)



$$x_i = X_i, \quad 1 \le i \le m$$

$$\text{net}_i = \sum_{j=1}^{i-1} W_{ij} x_i, \quad m < i \le N + n$$

$$x_i = s(\text{net}_i), \quad m < i \le N + n$$

$$Y_i = x_{i+N}, \quad 1 \le i \le n$$

Learning Rule

- Objective/Error Function

$$E = \sum_{t=1}^{T} E(t) = \sum_{t=1}^{T} \sum_{i=1}^{n} \frac{1}{2} \left(\widehat{Y}_i(t) - Y_i(t) \right)^2$$

- Target
  - $argmin_{\boldsymbol{W}}\, E$
- Approach: Back Propagation
  - Calculate derivatives of *E* with respect to all the weights (i.e. feedback to weights, F_W_{ij})
    - Core of Back Propagation: “Method of calculating derivatives exactly and efficiently in large systems”
  - Update the weights.
    - New $W_{ij} = W_{ij} - \eta * F_W_{ij}$

Back propagation

- Ordered Derivatives (F_z_i: Feedback to z_i)

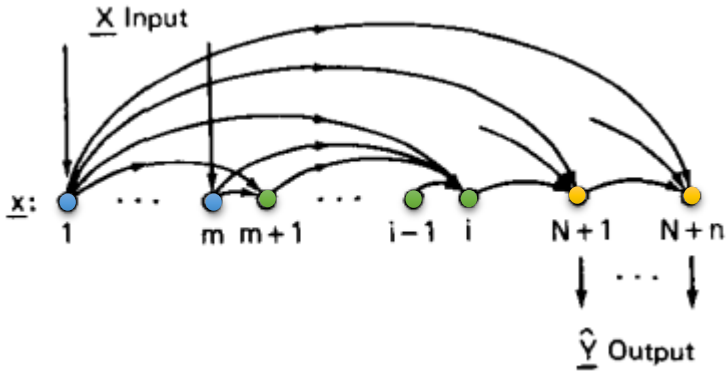
$$\frac{\partial^+ \text{TARGET}}{\partial z_i} = \frac{\partial \text{TARGET}}{\partial z_i} + \sum_{j>i} \frac{\partial^+ \text{TARGET}}{\partial z_j} * \frac{\partial z_j}{\partial z_i}$$

$$F_z_i = \frac{\partial E}{\partial z_i} + \sum_{j>i} F_z_i * \frac{\partial z_j}{\partial z_i}.$$

- Example
  - $z_2 = 4 * z_1$
  - $z_3 = 3 * z_1 + 5 * z_2$

Derivatives of z_3 w.r.t. z_1
Partial: 3
Ordered: 23

Back propagation

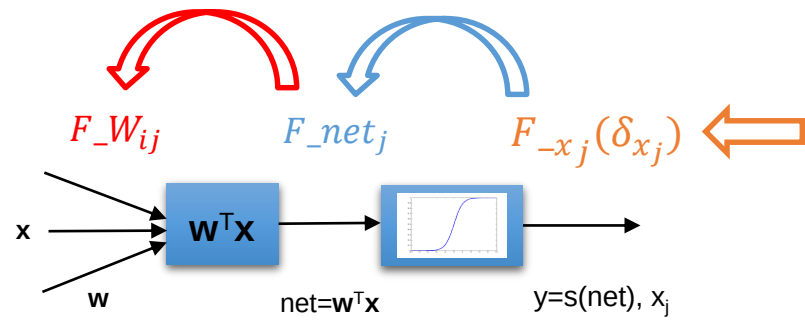


$$x_i = X_i, \quad 1 \le i \le m$$

$$\text{net}_i = \sum_{j=1}^{i-1} W_{ij} x_i, \quad m < i \le N + n$$

$$x_i = s(\text{net}_i), \quad m < i \le N + n$$

$$Y_i = x_{i+N}, \quad 1 \le i \le n$$



$$F_W_{ij} = \frac{\partial E}{\partial W_{ij}} = \frac{\partial E}{\partial net_j}\frac{\partial net_j}{\partial W_{ij}} = \frac{\partial E}{\partial net_j} x_i = F_net_j * x_i$$

$$F_net_j = \frac{\partial E}{\partial net_j} = \frac{\partial E}{\partial x_j}\frac{\partial x_j}{\partial net_j} = F_x_j * s'(net_j)$$

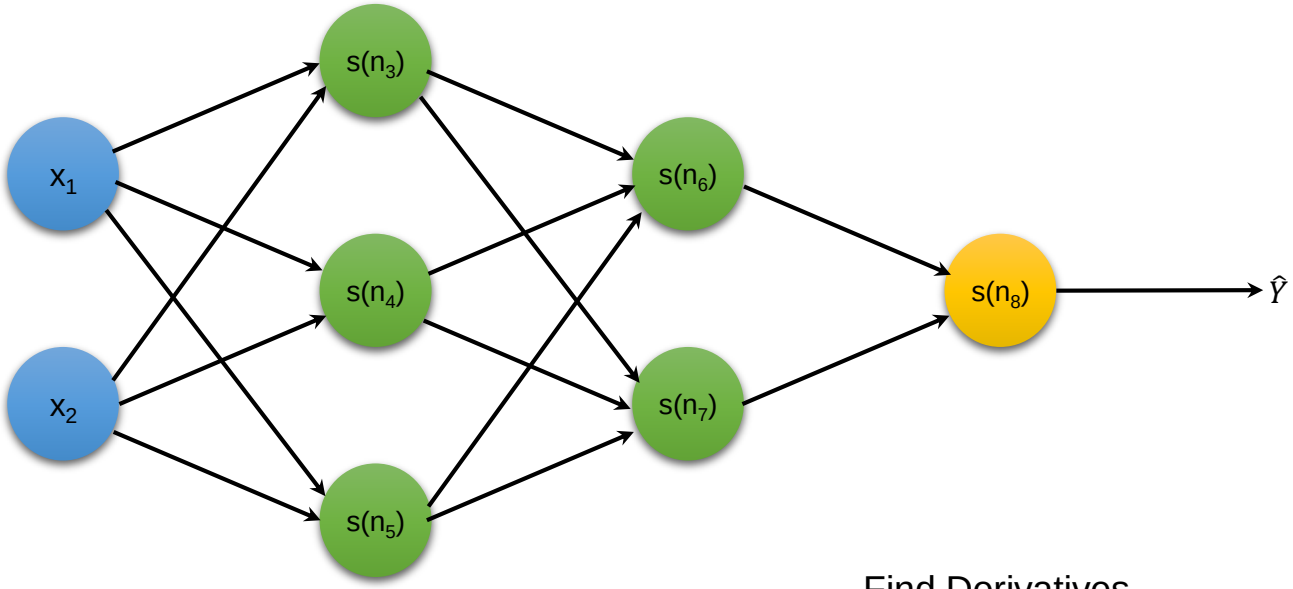
$$F_x_j = \frac{\partial E}{\partial x_j} = \frac{\partial E}{\partial \hat{Y}_{j-N}} + \sum_{k>j} \frac{\partial E}{\partial net_k}\frac{\partial net_k}{\partial x_k} = F_{-\hat{Y}_{j-N}} + \sum_{k>j} F_net_k * W_{jk} = \delta_{x_j}$$

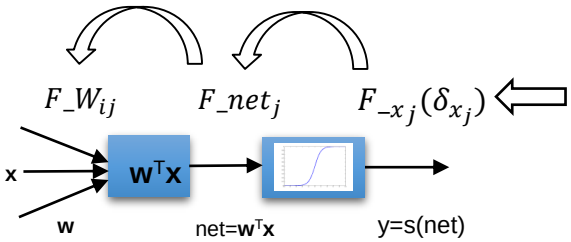
$$E = \sum_{t=1}^{T}\sum_{i=1}^{n} \frac{1}{2}\left(\hat{Y}_i(t) - Y_i(t)\right)^2$$

Remark:

$$F_{-\hat{Y}(t)} = \frac{\partial E}{\partial \hat{Y}_j(t)} = \begin{cases} \hat{Y}_j(t) - Y_j(t), & 1 \le j \le n \\ 0, & else \end{cases}$$

Back propagation





Find Derivatives

$F_W_{ij} = F_net_j * x_i$

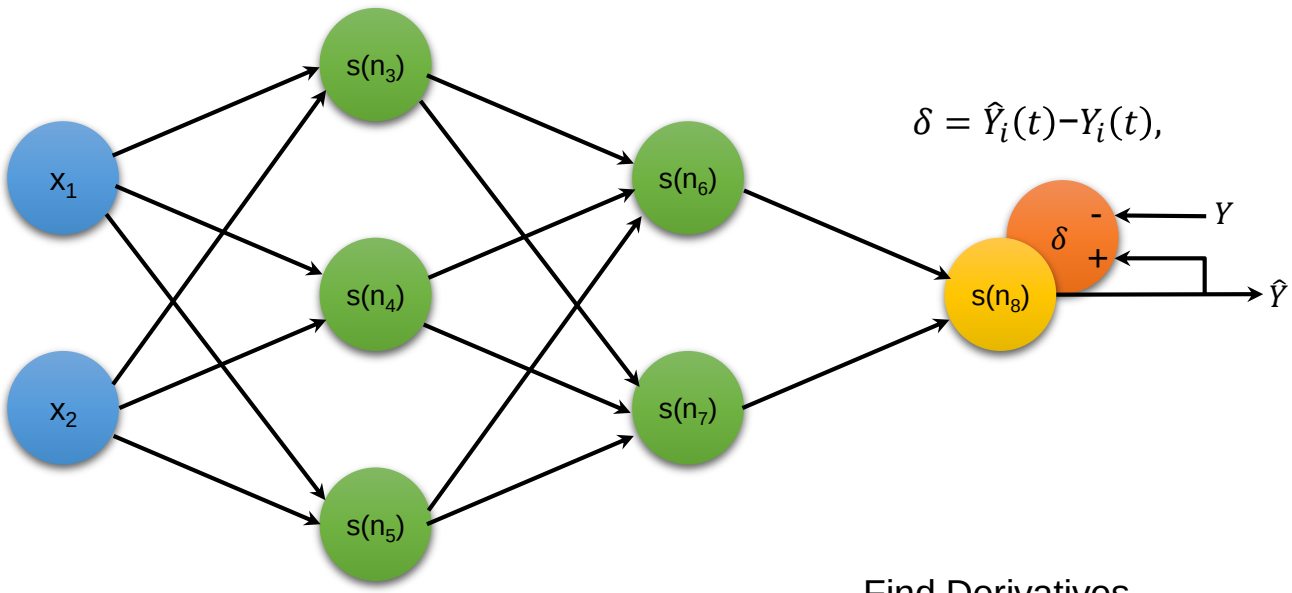
$F_net_j = F_x_j * s'(net_j)$

$F_x_j = F_\hat{Y}_{j-N} + \sum_{k>j} F_net_k * W_{jk} = \delta_{x_j}$

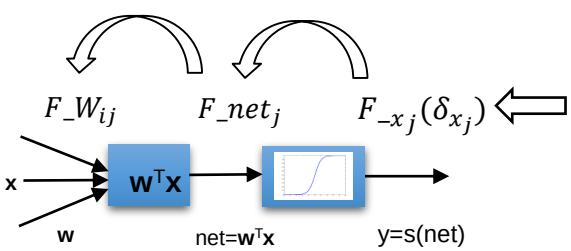
Update weights

$W'_{ij} = W_{ij} - \eta * F_W_{ij} = W_{ij} - \eta \delta_{x_j} s'(net_j) x_i$

Back propagation



$$\delta = \hat{Y}_i(t) - Y_i(t),$$



Find Derivatives

$$F_W_{ij} = F_net_j * x_i$$

$$F_net_j = F_x_j * s'(net_j)$$

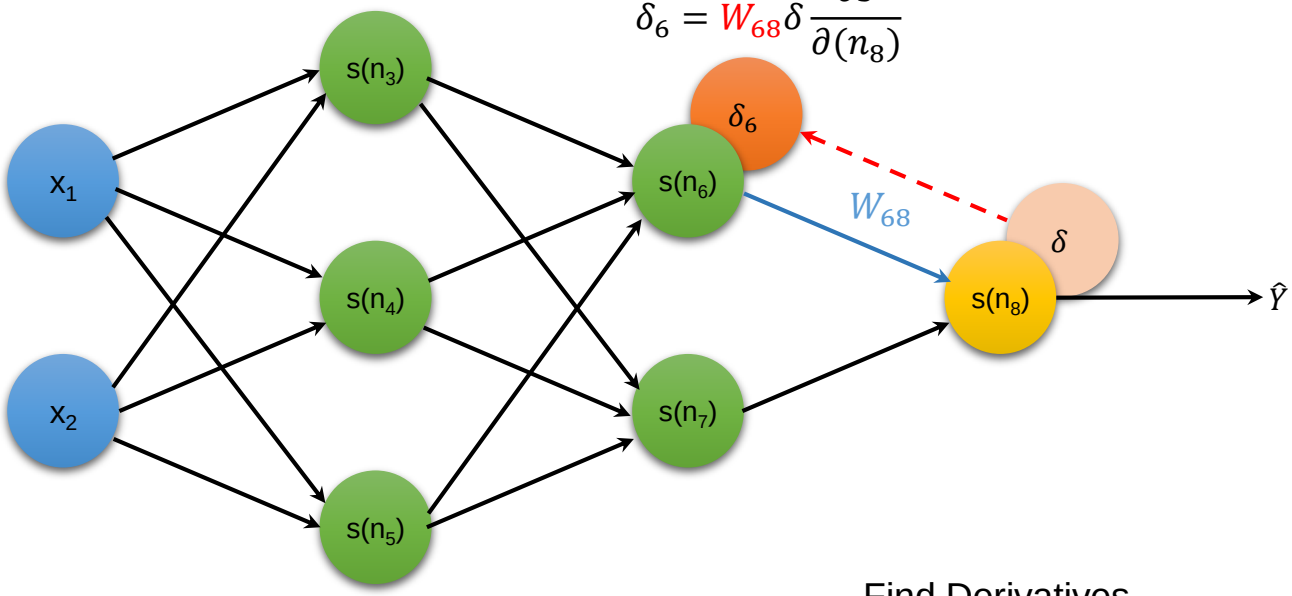
$$F_x_j = F_{_\hat{Y}_{j-N}} + \sum_{k>j} F_net_k * W_{jk} = \delta_{x_j}$$

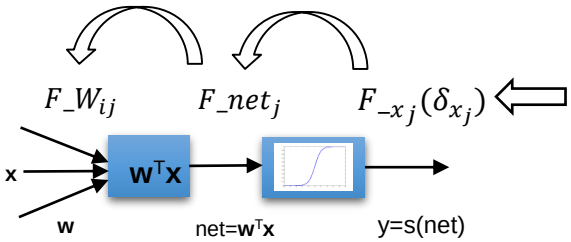
Update weights

$$W'_{ij} = W_{ij} - \eta * F_W_{ij} = W_{ij} - \eta \delta_{x_j} s'(net_j) x_i$$

Back propagation

$$\delta_6 = W_{68}\delta\frac{\partial s}{\partial(n_8)}$$





Find Derivatives

$F_W_{ij} = F_net_j * x_i$

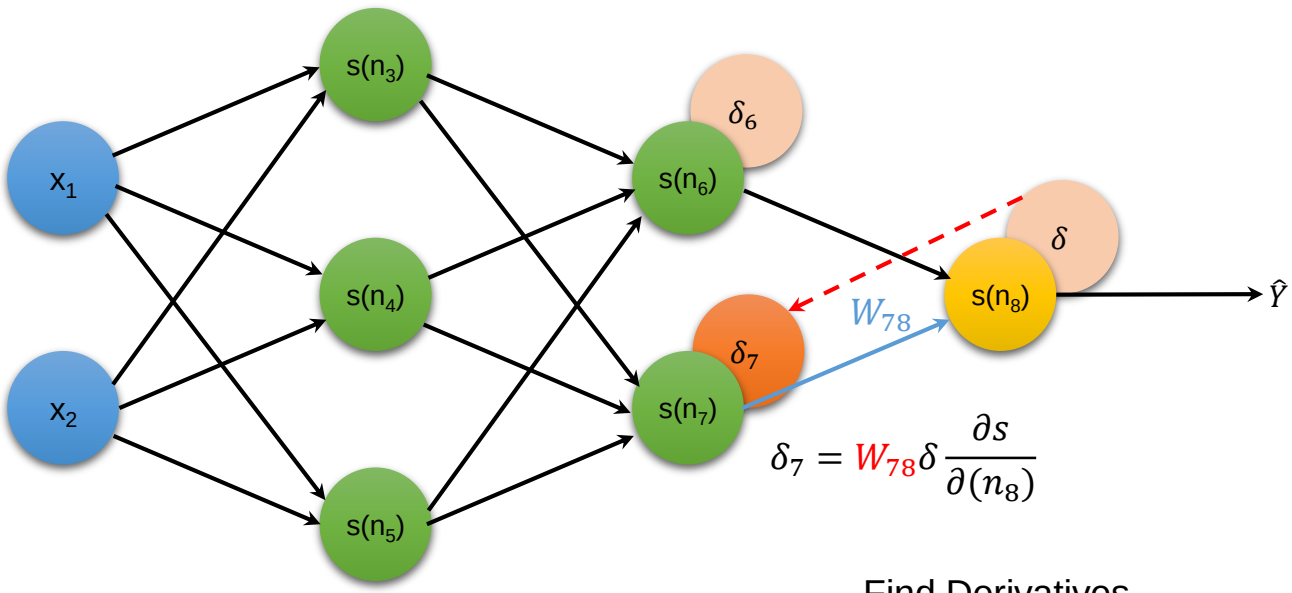
$F_net_j = F_x_j * s'(net_j)$

$F_x_j = F_\hat{Y}_{j-N} + \sum_{k>j} F_net_k * W_{jk} = \delta_{x_j}$

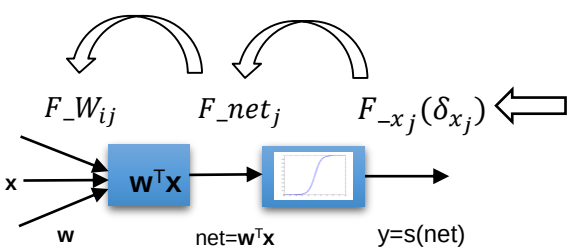
Update weights

$W'_{ij} = W_{ij} - \eta * F_W_{ij} = W_{ij} - \eta\delta_{x_j}s'(net_j)x_i$

Back propagation



$$\delta_7 = W_{78}\delta\frac{\partial s}{\partial(n_8)}$$



Find Derivatives

$F_W_{ij} = F_net_j * x_i$

$F_net_j = F_x_j * s'(net_j)$

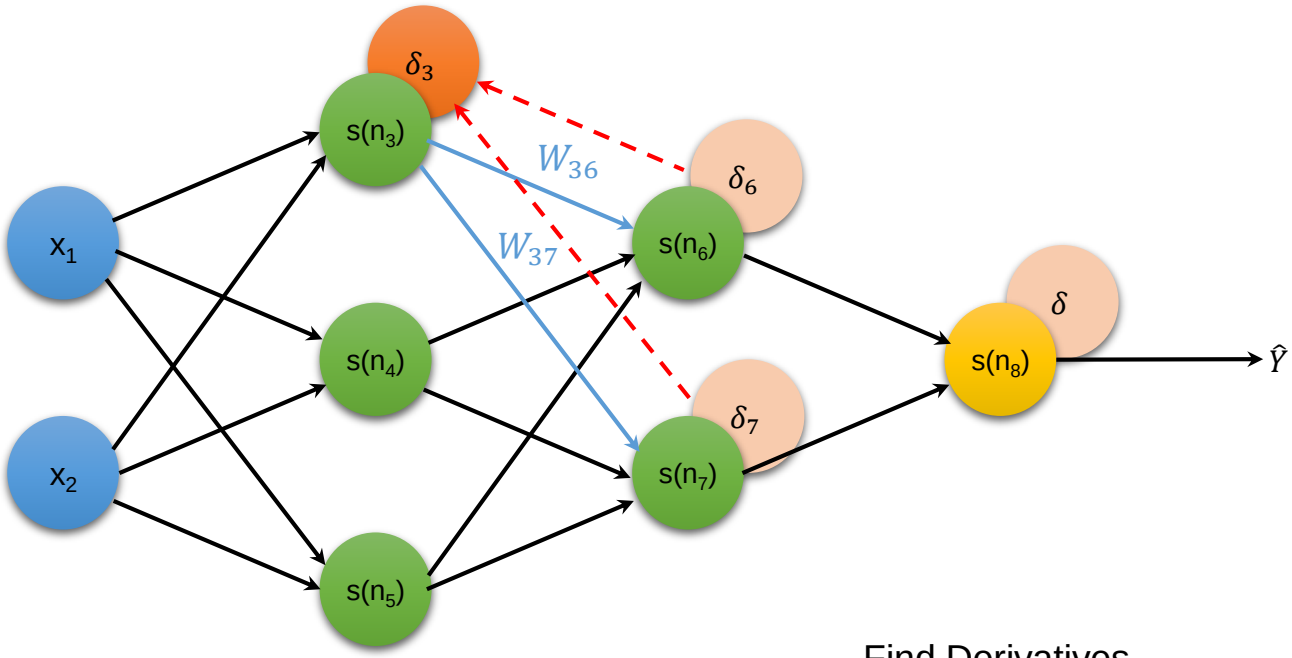
$F_x_j = F_\hat{Y}_{j-N} + \sum_{k>j} F_net_k * W_{jk} = \delta_{x_j}$

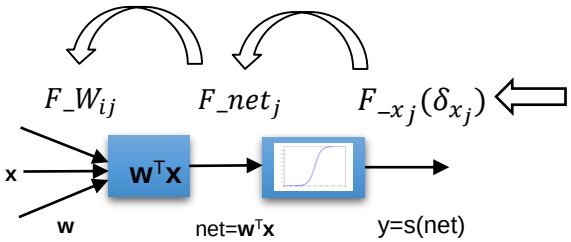
Update weights

$W'_{ij} = W_{ij} - \eta * F_W_{ij} = W_{ij} - \eta\delta_{x_j}s'(net_j)x_i$

Back propagation

$$\delta_3 = W_{36}\delta_6 \frac{\partial s}{\partial(n_6)} + W_{37}\delta_7 \frac{\partial s}{\partial(n_7)}$$





Find Derivatives

$F_W_{ij} = F_net_j * x_i$

$F_net_j = F_x_j * s'(net_j)$

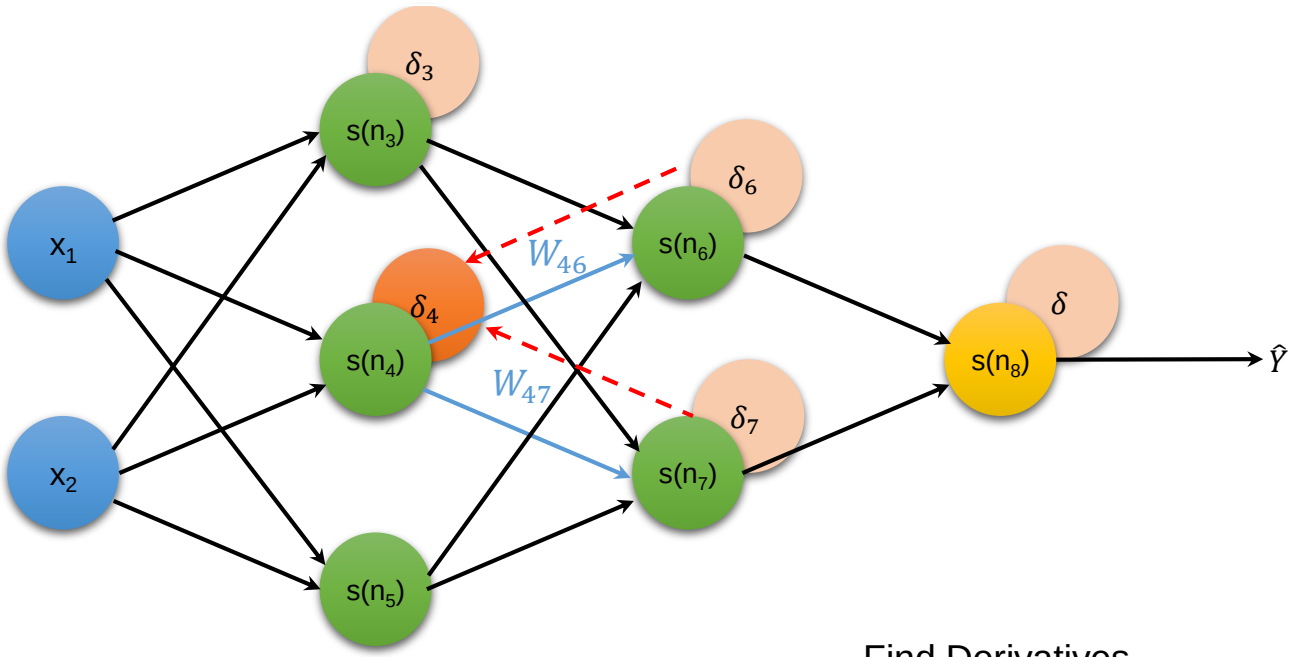
$F_x_j = F_\hat{Y}_{j-N} + \sum_{k>j} F_net_k * W_{jk} = \delta_{x_j}$

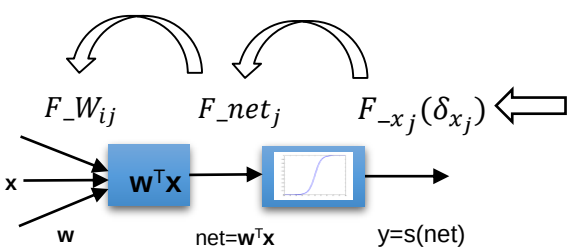
Update weights

$W'_{ij} = W_{ij} - \eta * F_W_{ij} = W_{ij} - \eta\delta_{x_j} s'(net_j) x_i$

Back propagation

$$\delta_4 = W_{46}\delta_6 \frac{\partial s}{\partial(n_6)} + W_{47}\delta_7 \frac{\partial s}{\partial(n_7)}$$





Find Derivatives

$F_W_{ij} = F_net_j * x_i$

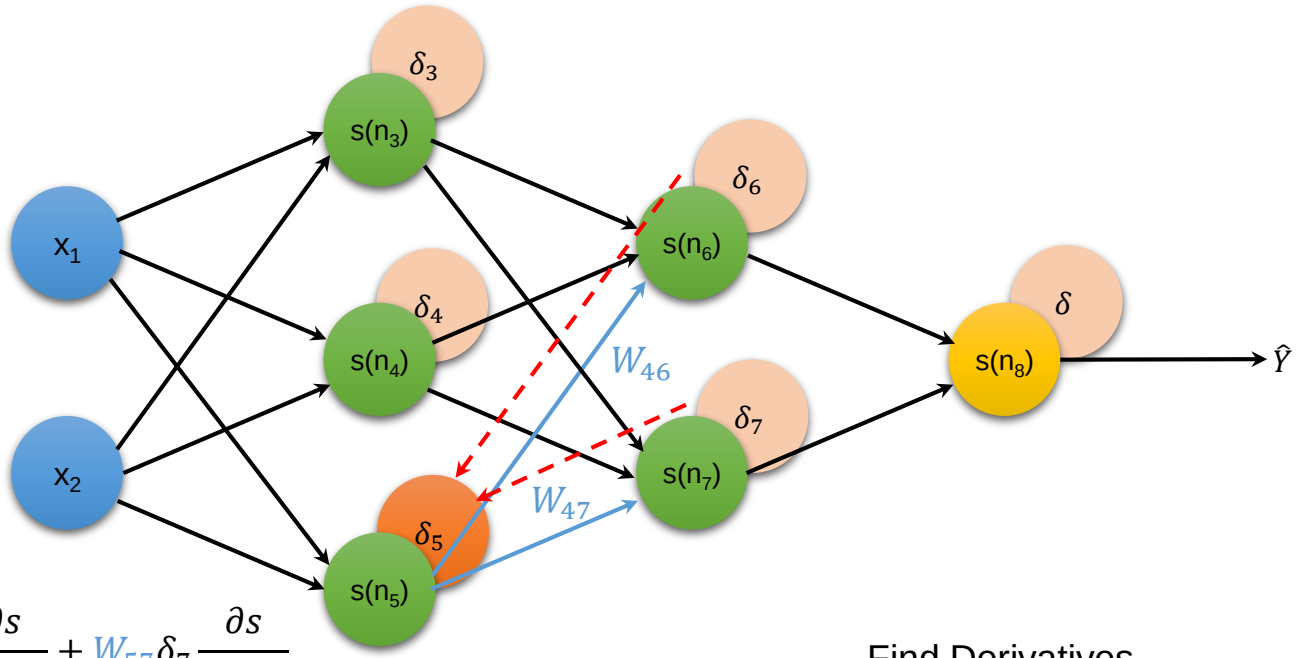
$F_net_j = F_x_j * s'(net_j)$

$F_x_j = F_\hat{Y}_{j-N} + \sum_{k>j} F_net_k * W_{jk} = \delta_{x_j}$

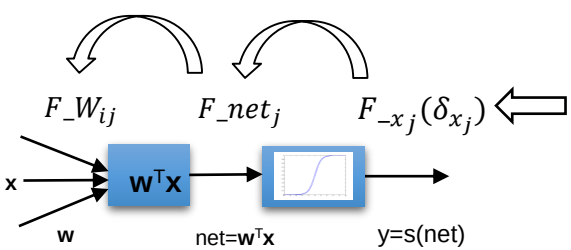
Update weights

$W'_{ij} = W_{ij} - \eta * F_W_{ij} = W_{ij} - \eta\delta_{x_j} s'(net_j) x_i$

Back propagation



$$\delta_5 = W_{56}\delta_6 \frac{\partial s}{\partial(n_6)} + W_{57}\delta_7 \frac{\partial s}{\partial(n_7)}$$



Find Derivatives

$F_W_{ij} = F_net_j * x_i$

$F_net_j = F_x_j * s'(net_j)$

$F_x_j = F_\hat{Y}_{j-N} + \sum_{k>j} F_net_k * W_{jk} = \delta_{x_j}$

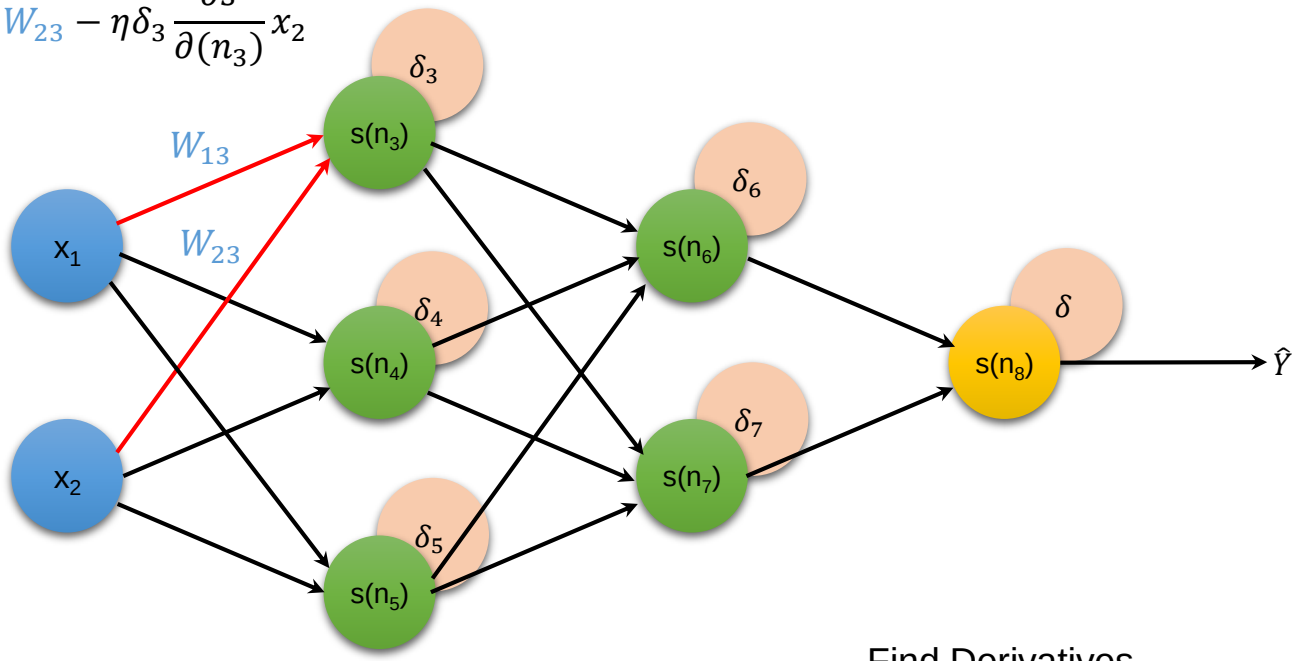
Update weights

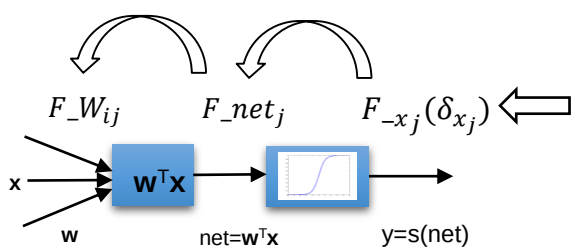
$W'_{ij} = W_{ij} - \eta * F_W_{ij} = W_{ij} - \eta\delta_{x_j} s'(net_j) x_i$

Back propagation

$$W'_{13} = W_{13} - \eta\delta_3 \frac{\partial s}{\partial(n_3)} x_1$$

$$W'_{23} = W_{23} - \eta\delta_3 \frac{\partial s}{\partial(n_3)} x_2$$





Find Derivatives

$F_W_{ij} = F_net_j * x_i$

$F_net_j = F_x_j * s'(net_j)$

$F_x_j = F_\hat{Y}_{j-N} + \sum_{k>j} F_net_k * W_{jk} = \delta_{x_j}$

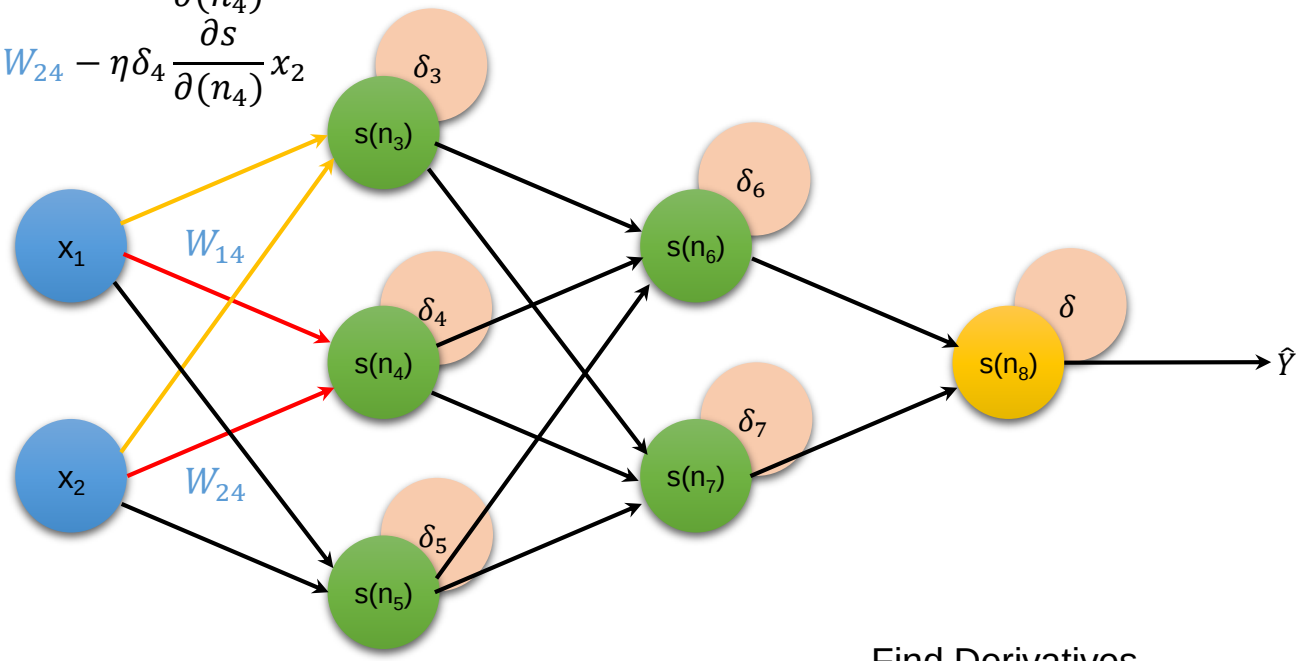
Update weights

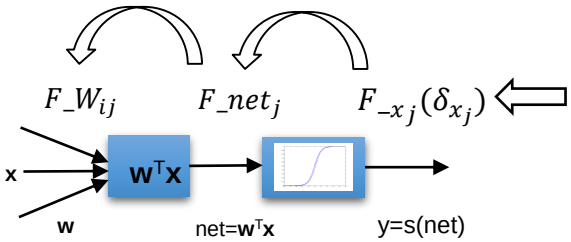
$W'_{ij} = W_{ij} - \eta * F_W_{ij} = W_{ij} - \eta\delta_{x_j} s'(net_j) x_i$

Back propagation

$$W'_{14} = W_{14} - \eta\delta_4 \frac{\partial s}{\partial(n_4)} x_1$$

$$W'_{24} = W_{24} - \eta\delta_4 \frac{\partial s}{\partial(n_4)} x_2$$





Find Derivatives

$F_W_{ij} = F_net_j * x_i$

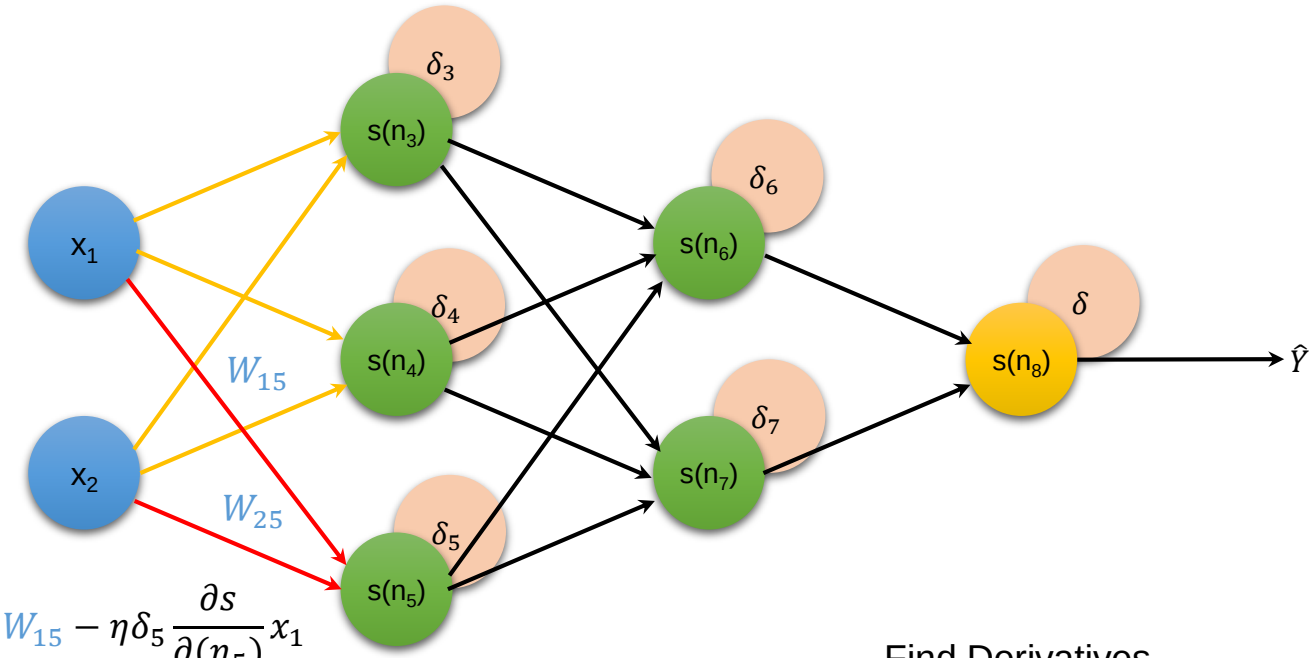
$F_net_j = F_x_j * s'(net_j)$

$F_x_j = F_\hat{Y}_{j-N} + \sum_{k>j} F_net_k * W_{jk} = \delta_{x_j}$

Update weights

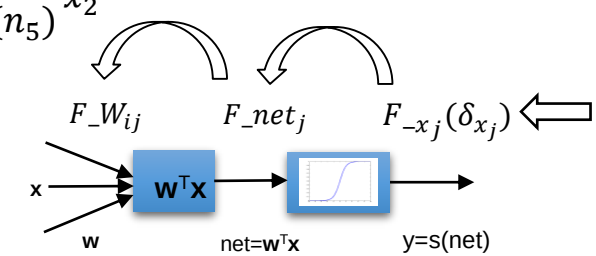
$W'_{ij} = W_{ij} - \eta * F_W_{ij} = W_{ij} - \eta\delta_{x_j} s'(net_j) x_i$

Back propagation



$$W'_{15} = W_{15} - \eta\delta_5 \frac{\partial s}{\partial(n_5)} x_1$$

$$W'_{25} = W_{25} - \eta\delta_5 \frac{\partial s}{\partial(n_5)} x_2$$



Find Derivatives

$F_W_{ij} = F_net_j * x_i$

$F_net_j = F_x_j * s'(net_j)$

$F_x_j = F_{_\hat{Y}_{j-N}} + \sum_{k>j} F_net_k * W_{jk} = \delta_{x_j}$

Update weights

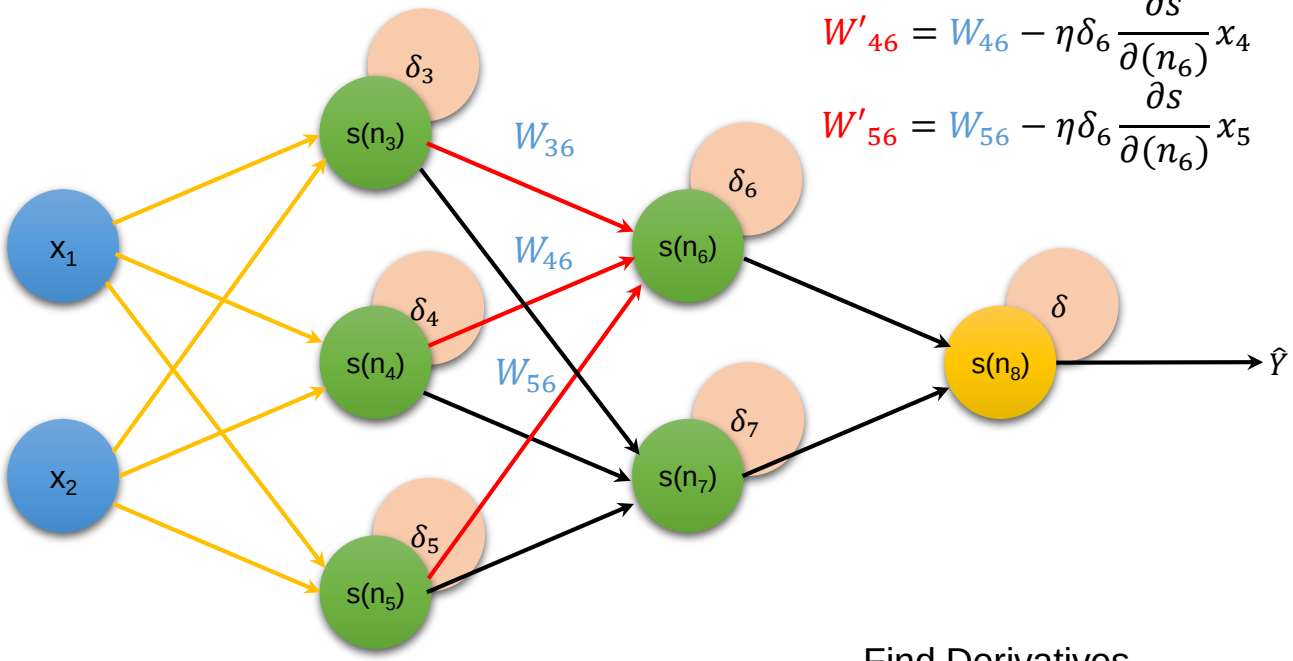
$W'_{ij} = W_{ij} - \eta * F_W_{ij} = W_{ij} - \eta\delta_{x_j} s'(net_j) x_i$

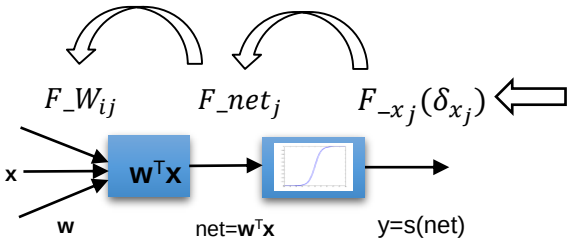
Back propagation

$$W'_{36} = W_{36} - \eta\delta_6 \frac{\partial s}{\partial(n_6)} x_3$$

$$W'_{46} = W_{46} - \eta\delta_6 \frac{\partial s}{\partial(n_6)} x_4$$

$$W'_{56} = W_{56} - \eta\delta_6 \frac{\partial s}{\partial(n_6)} x_5$$





Find Derivatives

$F_W_{ij} = F_net_j * x_i$

$F_net_j = F_x_j * s'(net_j)$

$F_x_j = F_\hat{Y}_{j-N} + \sum_{k>j} F_net_k * W_{jk} = \delta_{x_j}$

Update weights

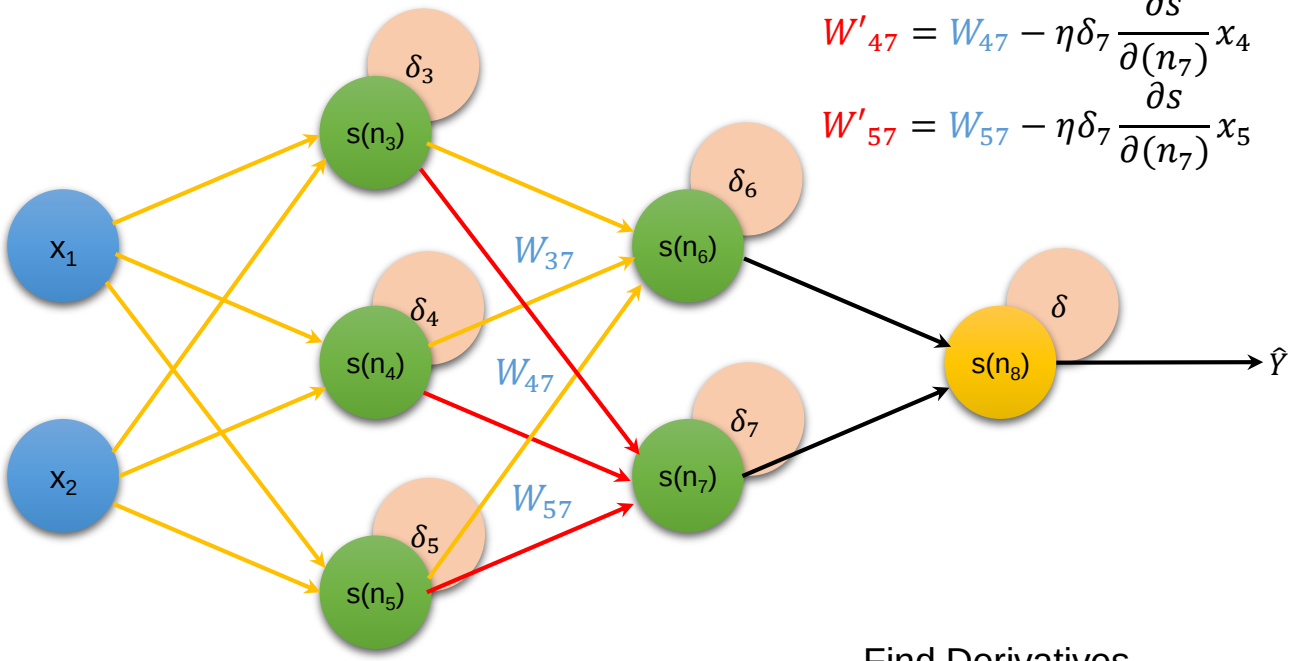
$W'_{ij} = W_{ij} - \eta * F_W_{ij} = W_{ij} - \eta\delta_{x_j} s'(net_j) x_i$

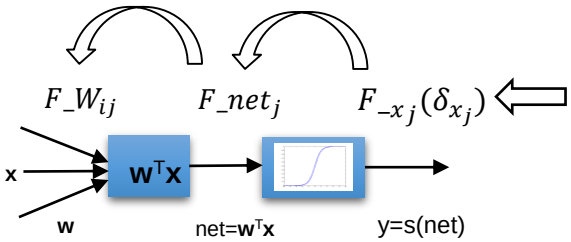
Back propagation

$$W'_{37} = W_{37} - \eta\delta_7 \frac{\partial s}{\partial(n_7)} x_3$$

$$W'_{47} = W_{47} - \eta\delta_7 \frac{\partial s}{\partial(n_7)} x_4$$

$$W'_{57} = W_{57} - \eta\delta_7 \frac{\partial s}{\partial(n_7)} x_5$$





Find Derivatives

$F_W_{ij} = F_net_j * x_i$

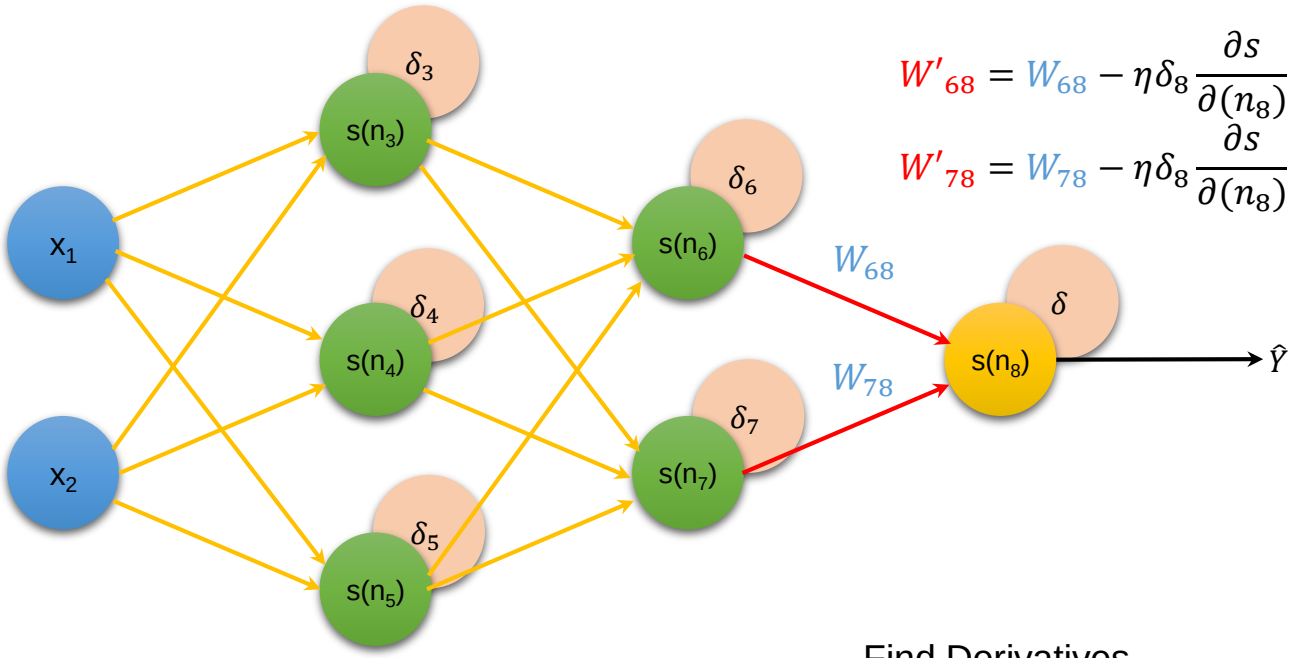
$F_net_j = F_x_j * s'(net_j)$

$F_x_j = F_{_\hat{Y}_{j-N}} + \sum_{k>j} F_net_k * W_{jk} = \delta_{x_j}$

Update weights

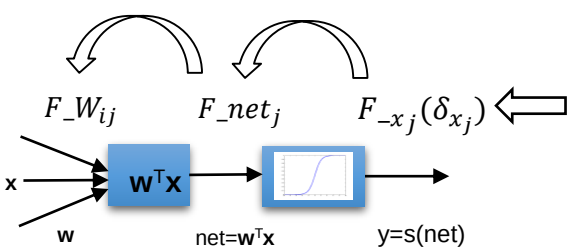
$W'_{ij} = W_{ij} - \eta * F_W_{ij} = W_{ij} - \eta\delta_{x_j} s'(net_j) x_i$

Back propagation



$$W'_{68} = W_{68} - \eta\delta_8 \frac{\partial s}{\partial(n_8)} x_6$$

$$W'_{78} = W_{78} - \eta\delta_8 \frac{\partial s}{\partial(n_8)} x_7$$



Find Derivatives

$F_W_{ij} = F_net_j * x_i$

$F_net_j = F_x_j * s'(net_j)$

$F_x_j = F_\hat{Y}_{j-N} + \sum_{k>j} F_net_k * W_{jk} = \delta_{x_j}$

Update weights

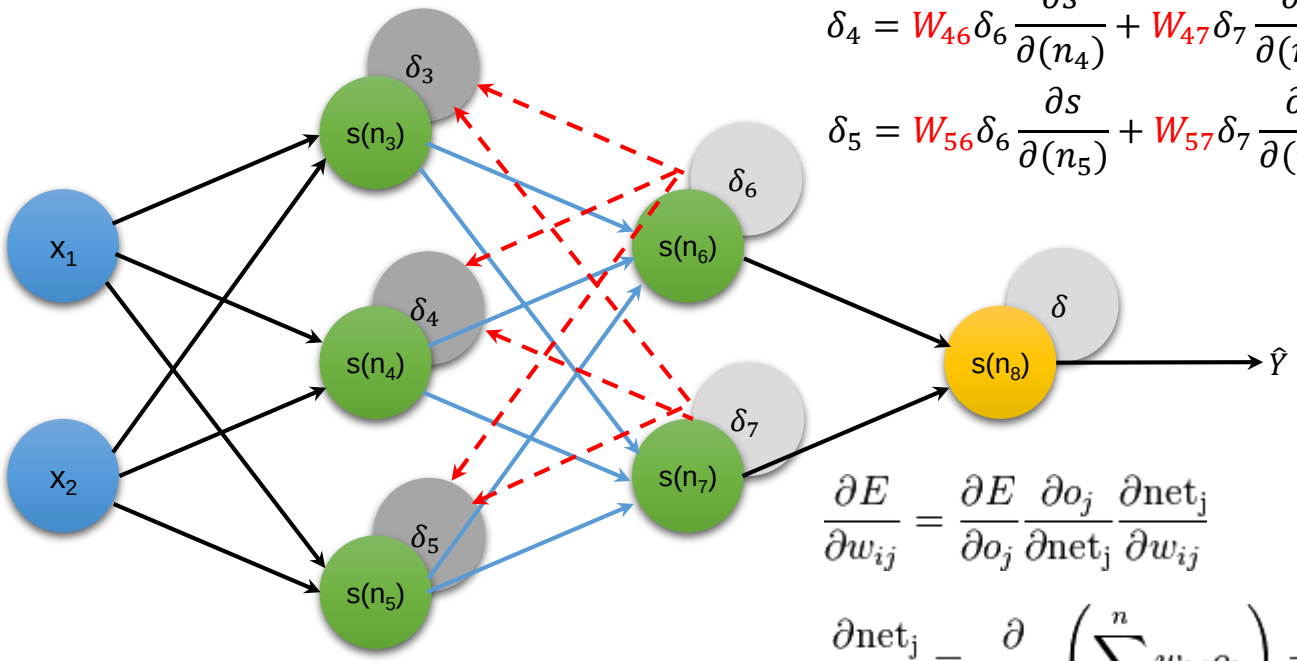
$W'_{ij} = W_{ij} - \eta * F_W_{ij} = W_{ij} - \eta\delta_{x_j} s'(net_j) x_i$

Back Propagation:Implementation

$$\delta_3 = W_{36}\delta_6 \frac{\partial s}{\partial(n_3)} + W_{37}\delta_7 \frac{\partial s}{\partial(n_3)}$$

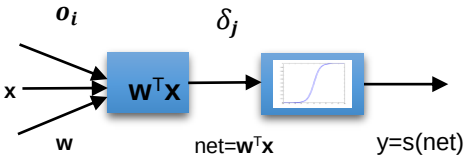
$$\delta_4 = W_{46}\delta_6 \frac{\partial s}{\partial(n_4)} + W_{47}\delta_7 \frac{\partial s}{\partial(n_4)}$$

$$\delta_5 = W_{56}\delta_6 \frac{\partial s}{\partial(n_5)} + W_{57}\delta_7 \frac{\partial s}{\partial(n_5)}$$



$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial o_j}\frac{\partial o_j}{\partial \mathrm{net_j}}\frac{\partial \mathrm{net_j}}{\partial w_{ij}}$$

$$\frac{\partial \mathrm{net_j}}{\partial w_{ij}} = \frac{\partial}{\partial w_{ij}}\left(\sum_{k=1}^{n} w_{kj} o_k\right) = o_i$$

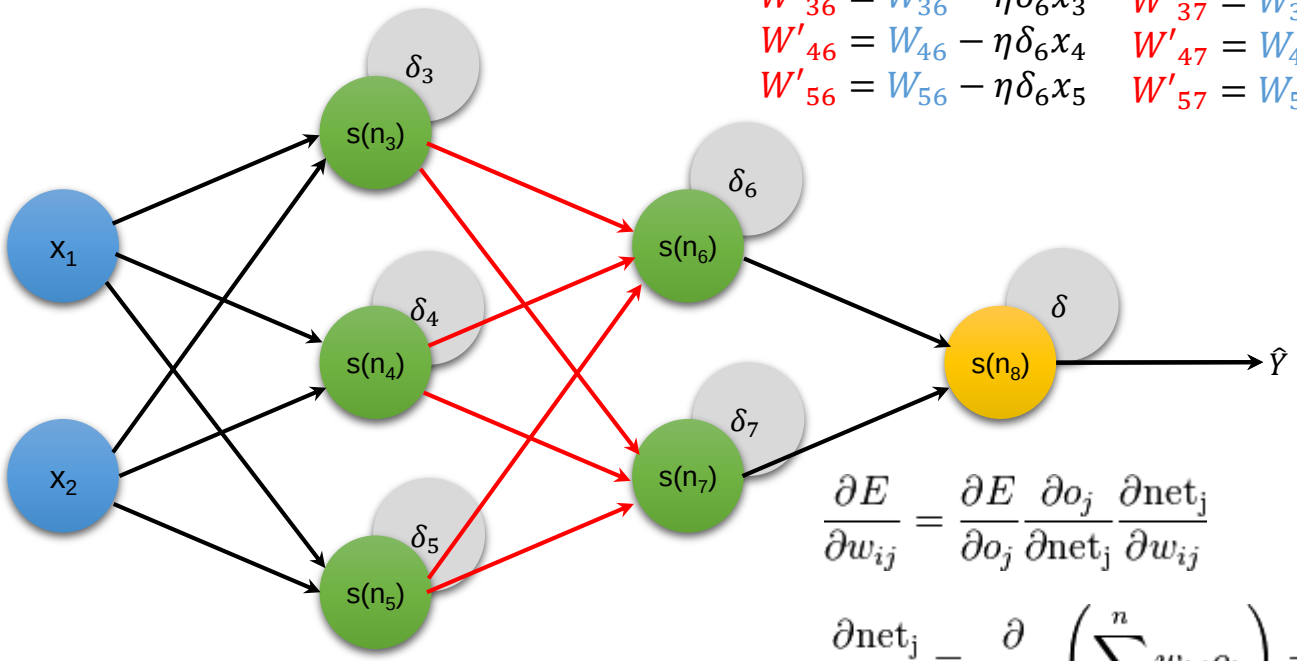


$$\frac{\partial E}{\partial w_{ij}} = \delta_j o_i$$

$$\delta_j \begin{cases} \dfrac{\partial o_j}{\partial \mathrm{net_j}} = \dfrac{\partial}{\partial \mathrm{net_j}}\varphi(\mathrm{net_j}) = \varphi(\mathrm{net_j})(1-\varphi(\mathrm{net_j})) \\ \dfrac{\partial E}{\partial o_j} = \displaystyle\sum_{l\in L}\left(\frac{\partial E}{\partial \mathrm{net}_l}\frac{\partial \mathrm{net}_l}{\partial o_j}\right) = \sum_{l\in L}\left(\frac{\partial E}{\partial o_l}\frac{\partial o_l}{\partial \mathrm{net}_l} w_{jl}\right) \end{cases}$$

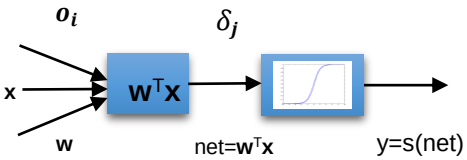
Back Propagation:Implementation

$$W'_{36} = W_{36} - \eta\delta_6 x_3 \qquad W'_{37} = W_{37} - \eta\delta_7 x_3$$
$$W'_{46} = W_{46} - \eta\delta_6 x_4 \qquad W'_{47} = W_{47} - \eta\delta_7 x_4$$
$$W'_{56} = W_{56} - \eta\delta_6 x_5 \qquad W'_{57} = W_{57} - \eta\delta_7 x_5$$



$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial o_j}\frac{\partial o_j}{\partial \mathrm{net_j}}\frac{\partial \mathrm{net_j}}{\partial w_{ij}}$$

$$\frac{\partial \mathrm{net_j}}{\partial w_{ij}} = \frac{\partial}{\partial w_{ij}}\left(\sum_{k=1}^{n} w_{kj} o_k\right) = o_i$$



$$\frac{\partial E}{\partial w_{ij}} = \delta_j o_i$$

$$\delta_j \begin{cases} \dfrac{\partial o_j}{\partial \mathrm{net_j}} = \dfrac{\partial}{\partial \mathrm{net_j}}\varphi(\mathrm{net_j}) = \varphi(\mathrm{net_j})(1-\varphi(\mathrm{net_j})) \\[2ex] \dfrac{\partial E}{\partial o_j} = \displaystyle\sum_{l\in L}\left(\frac{\partial E}{\partial \mathrm{net}_l}\frac{\partial \mathrm{net}_l}{\partial o_j}\right) = \sum_{l\in L}\left(\frac{\partial E}{\partial o_l}\frac{\partial o_l}{\partial \mathrm{net}_l} w_{jl}\right) \end{cases}$$

What about *t* ?

Pattern Learning

- Until satisfied do:
- For each training data t=1:T
  - ❖ Compute network ouput with feed forward algorithm
  - ❖ For each ouput i=1:n
    - ➢ Compute $F_\hat{Y}(t) = \frac{\partial E}{\partial \hat{Y}_j(t)} = \hat{Y}_j(t) - Y_j(t)$
  - ❖ For each neurons in hidden layer
    - ➢ Compute F_W_{ij} with backpropogation
  - ❖ For all weights
    - ➢ Update $W'_{ij} = W_{ij} - \eta * F_W_{ij}$

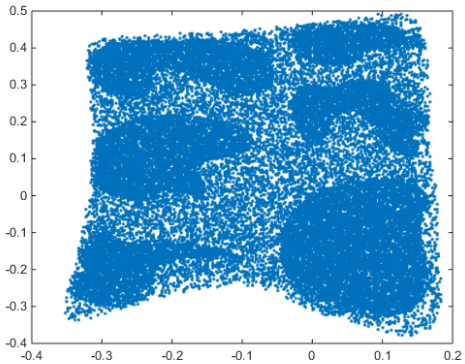
Batch Learning

- Until satisfied do:
- For each training data t=1:T
  - ❖ Compute network ouput with feed forward algorithm
  - ❖ For each ouput i=1:n
    - ➢ Compute $F_\hat{Y}(t) = \frac{\partial E}{\partial \hat{Y}_j(t)} = \hat{Y}_j(t) - Y_j(t)$
  - ❖ For each neurons in hidden layer
    - ➢ Compute F_W_{ij} with backpropogation
- For all weights
  - ❖ Update $W'_{ij} = W_{ij} - \eta * F_{W_{ij}}$

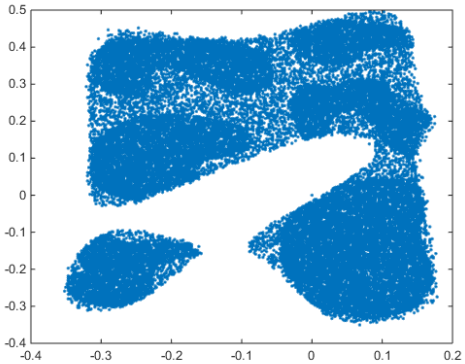
On-line learning => pattern learning/realtime learning

Online vs Batch

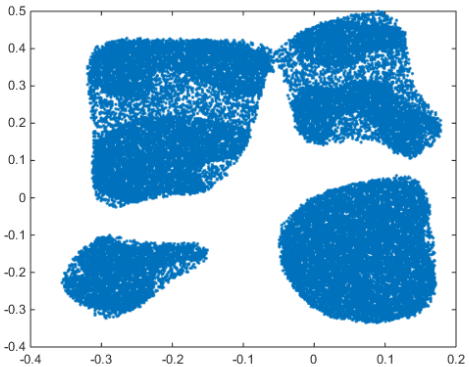
- Time vs Accuracy (1000 epochs, 3 hidden layers)
  - Online: 632 secs
  - Batch: 87 secs
  - Modern way: mini-batch (mSGD)
    - Divide data into mini-batches



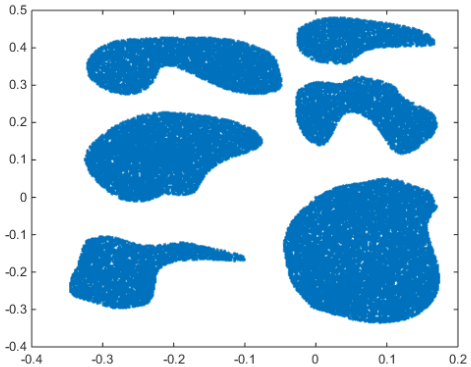
Size: 50000,
t= 87 sec



Size: 2000,
T = 158 sec

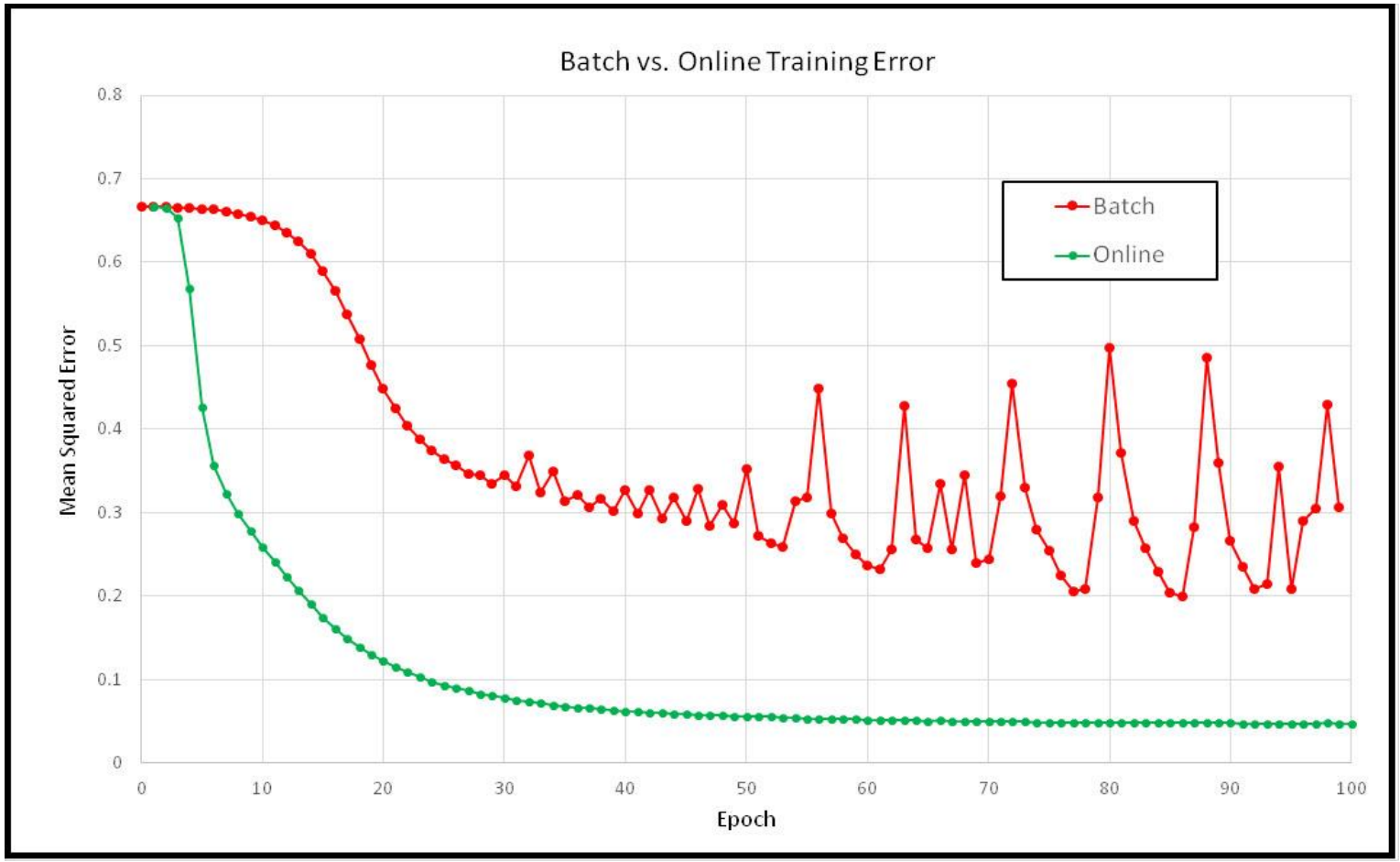


Size: 100,
T = 223 sec



Size: 1,
T = 632 sec

Online vs Batch

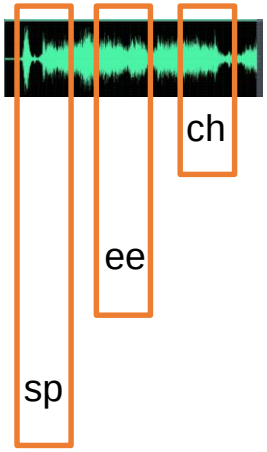


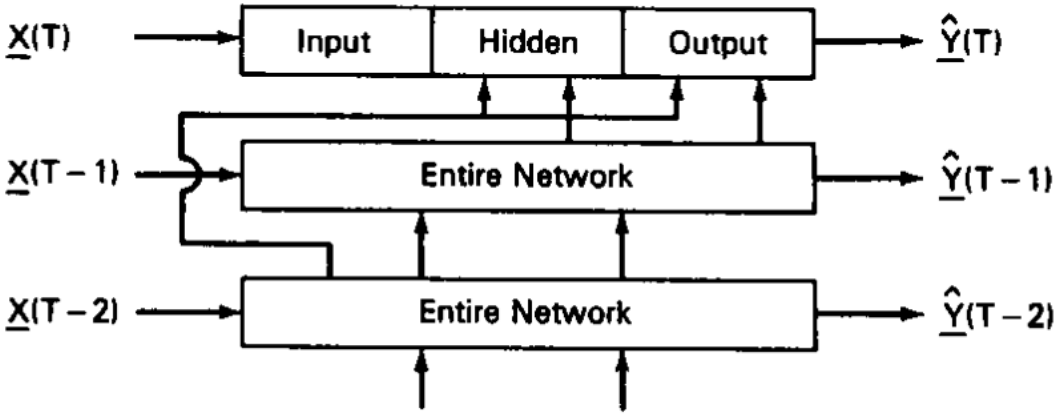
First 100 epochs of training on the Iris Data Set. (Also note over-fitting of training data…)
"The General Inefficiency of Bach Training for Gradient Descent Learning" by D. Wilson and T. Martine, 2003

Back Propagation Through Time

- Use previous "memory" to improve classification
  - Every neuron at time t is allowed to input values from any of the neurons at time t-1 and t-2
  - Example: speech recognition

X(t) = 'ch'
Spee'ch' or Fren'ch'

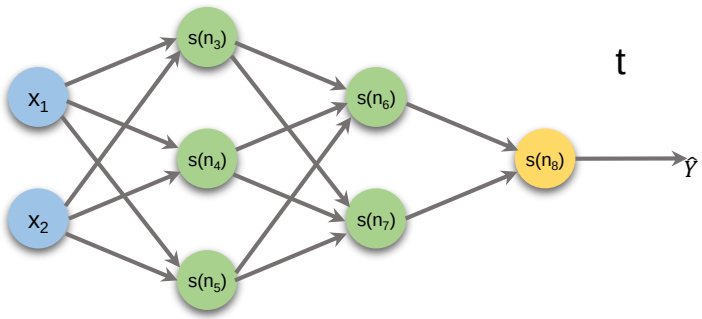




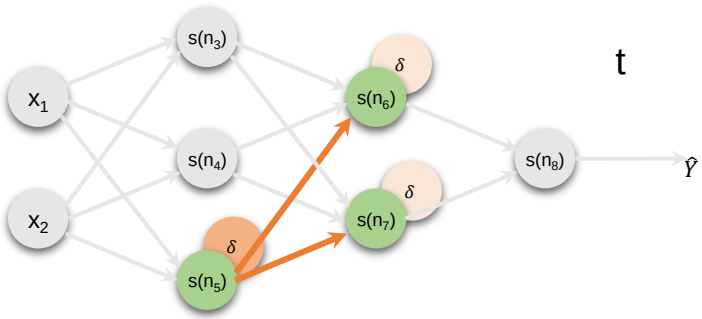
Label:
speech
speech
speech

$$\text{net}_i(t) = \sum_{j=1}^{i-1} W_{ij} x_j(t) + \sum_{j=1}^{N+n} W'_{ij} x_j(t-1) + \sum_{j=1}^{N+n} W''_{ij} x_j(t-2).$$

BPTT



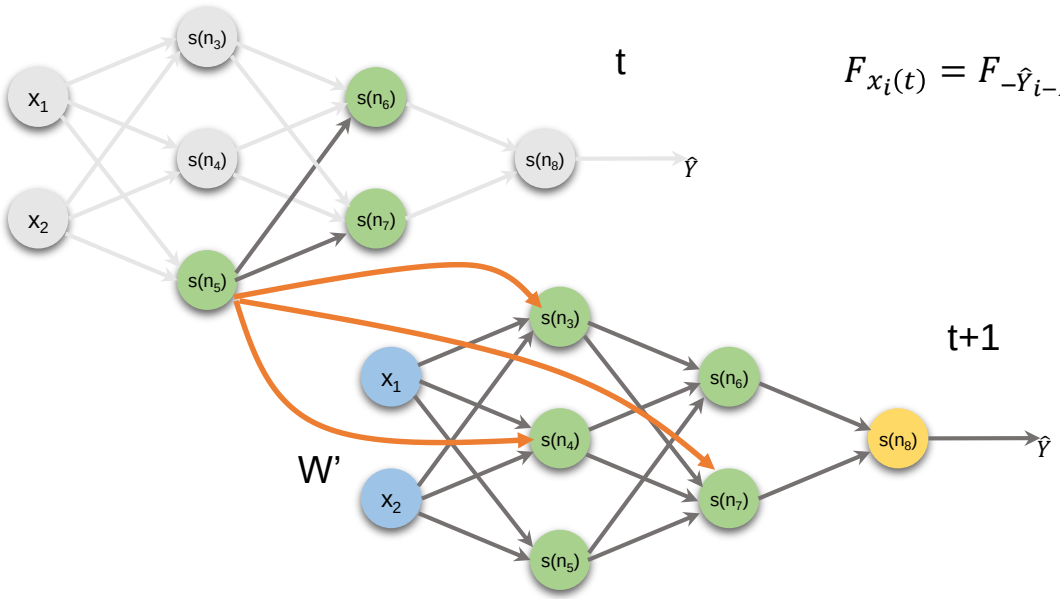
BPTT



$$F_{x_i(t)} = F_{-\hat{Y}_{i-N}} + \sum_{j>i} W_{ji} F_{net_j}(t)$$

$$net_i(t) = \sum_{j=1}^{i-1} W_{ij} x_j(t)$$

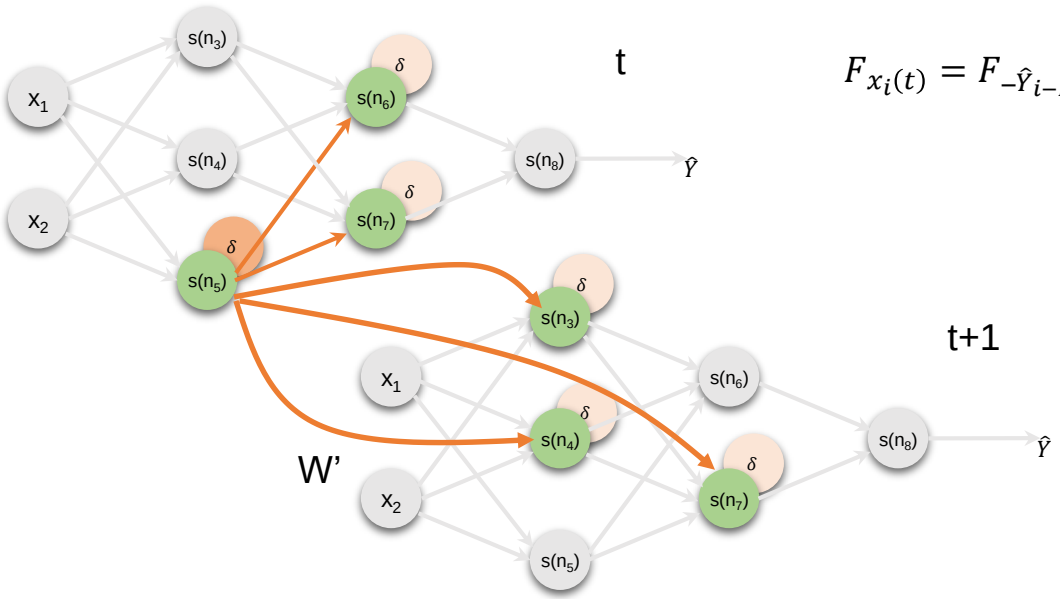
BPTT



$$F_{x_i(t)} = F_{-\hat{Y}_{i-N}} + \sum_{j>i} W_{ji} F_{net_j}(t) + \sum_{j>i} W'_{ji} F_{net_j}(t+1)$$

$$net_i(t+1) = \sum_{j=1}^{i-1} W'_{ij} x_j(t+1) + \sum_{j=1}^{i-1} W_{ij} x_j(t)$$

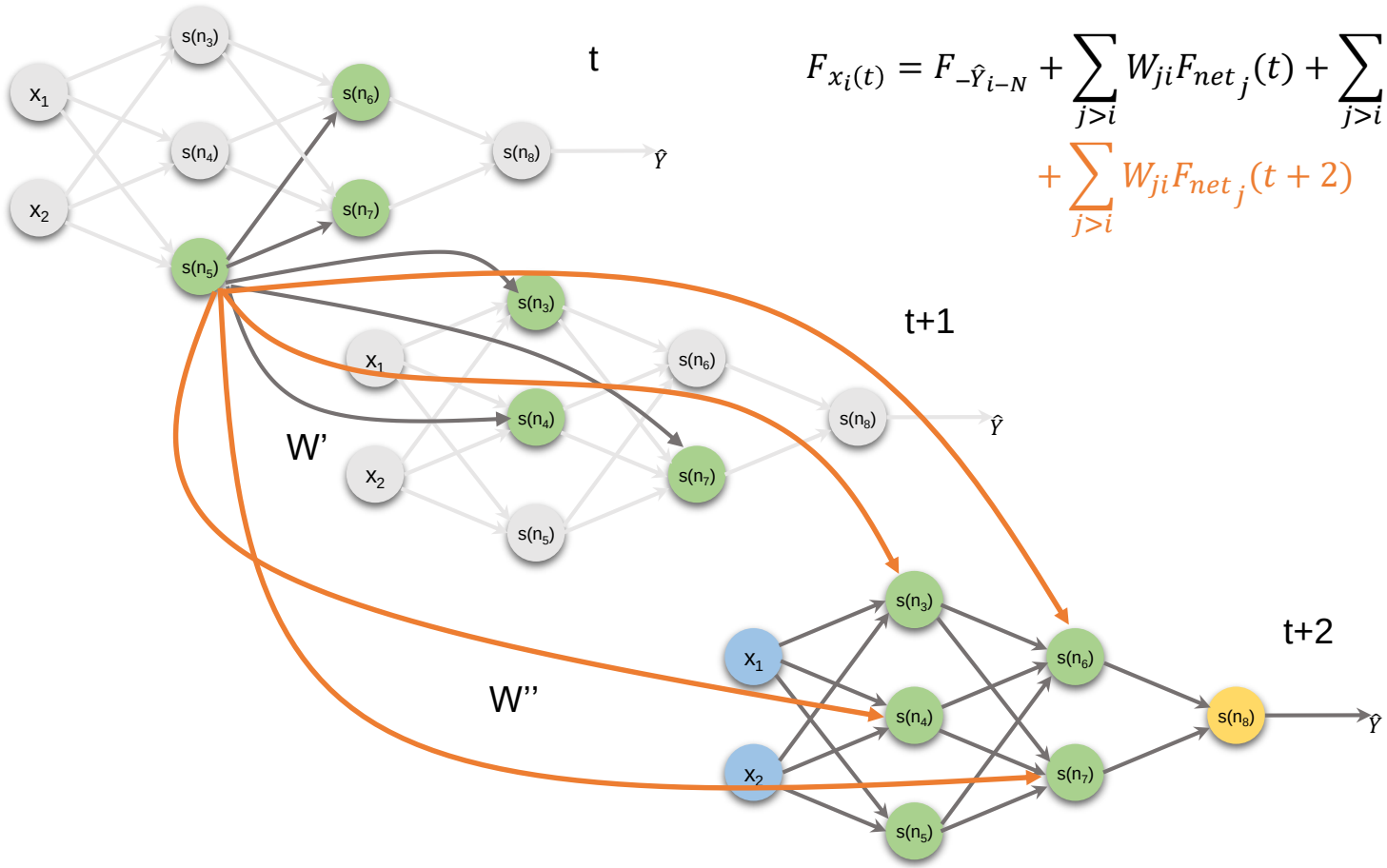
BPTT



$$F_{x_i(t)} = F_{-\hat{Y}_{i-N}} + \sum_{j>i} W_{ji} F_{net_j}(t) + \sum_{j>i} W'_{ji} F_{net_j}(t+1)$$

$$net_i(t+1) = \sum_{j=1}^{i-1} W'_{ij} x_j(t+1) + \sum_{j=1}^{i-1} W_{ij} x_j(t)$$

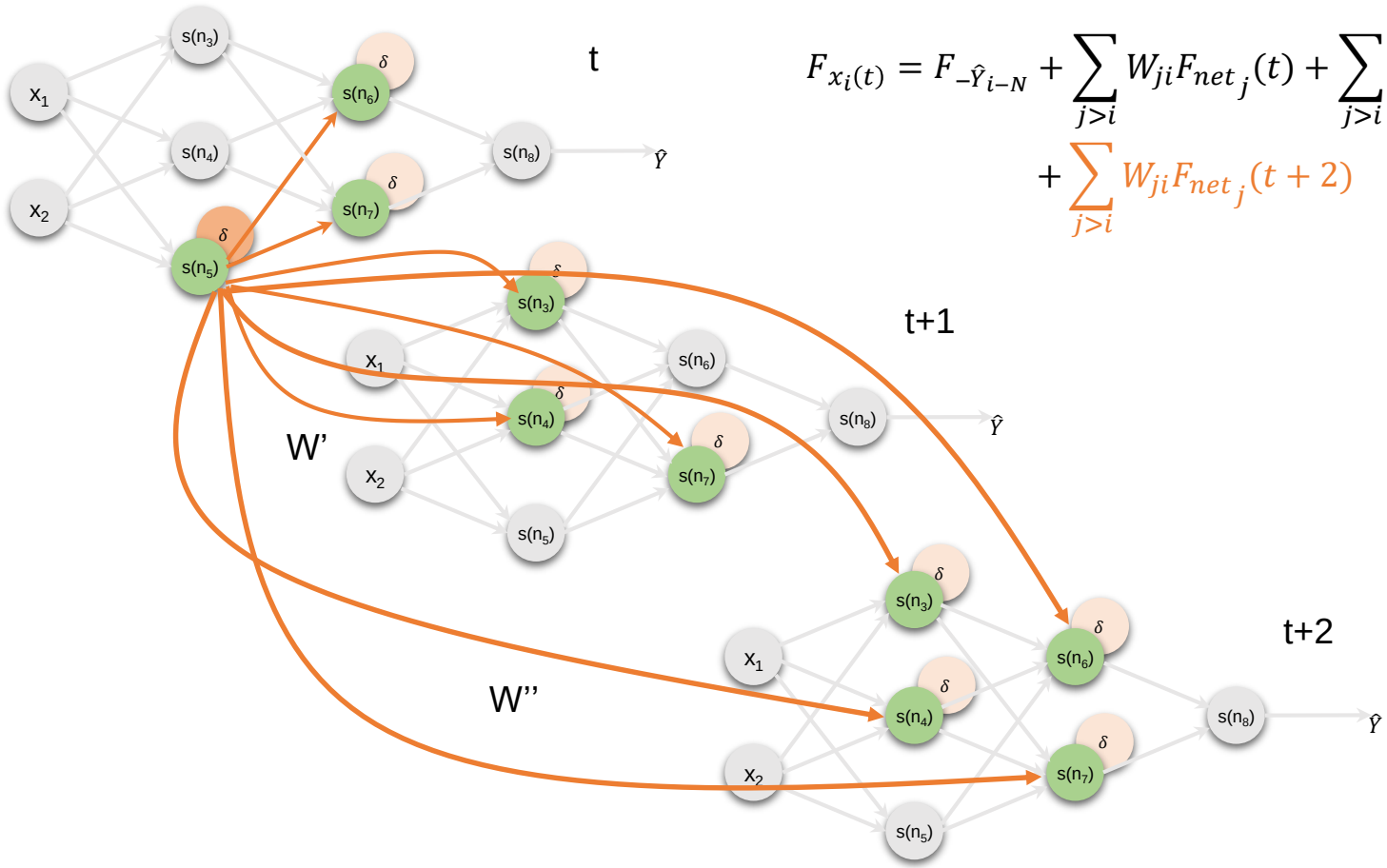
BPTT



$$F_{x_i(t)} = F_{-\hat{Y}_{i-N}} + \sum_{j>i} W_{ji} F_{net_j}(t) + \sum_{j>i} W'_{ji} F_{net_j}(t+1) + \sum_{j>i} W_{ji} F_{net_j}(t+2)$$

$$net_i(t+2) = \sum_{j=1}^{i-1} W''_{ij} x_j(t+2) + \sum_{j=1}^{i-1} W'_{ij} x_j(t+1) + \sum_{j=1}^{i-1} W_{ij} x_j(t)$$

BPTT



$$F_{x_i(t)} = F_{-\hat{Y}_{i-N}} + \sum_{j>i} W_{ji} F_{net_j}(t) + \sum_{j>i} W'_{ji} F_{net_j}(t+1) + \sum_{j>i} W_{ji} F_{net_j}(t+2)$$

$$net_i(t+2) = \sum_{j=1}^{i-1} W''_{ij} x_j(t+2) + \sum_{j=1}^{i-1} W'_{ij} x_j(t+1) + \sum_{j=1}^{i-1} W_{ij} x_j(t)$$

BPTT Implementation details

- Required to store/compute lots of intermediate info
- Use smaller learning rates
- Initialize W’ , W’’ to zeros
- We might not care about errors all the time
- Choice of network topology
  - For a brain-like network (each neuron only receive input from a small number of other notes), use Linked-List
- Performance concerns
  - Avoid calculating exponential
  - Other nonlinear functions/Threshold instead

$$s(z) = 0, \quad z < 0$$
$$s(z) = z, \quad 0 < z < 1$$
$$s(z) = 1, \quad z > 1.$$

Conlcusion

- Contribution
  - A general network topology
  - A general and theoretic analysis of BP/BPTT learning theory
  - Pseudo codes are provided (though he didn't test them)
- Limitation
  - Few implementation details/know-how are provided
    - Eg. W initialization, learning rate adaption, range checking … etc

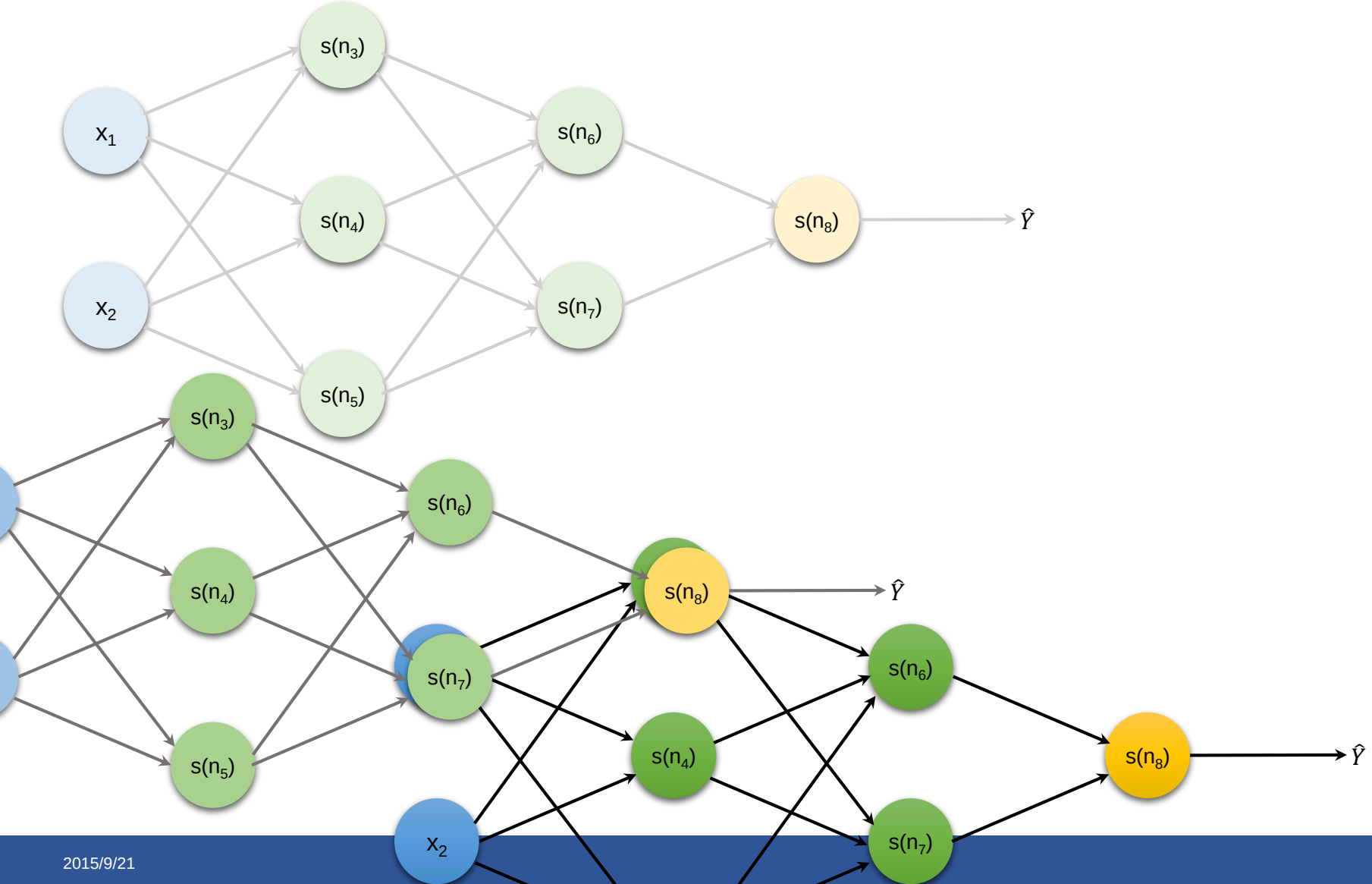
      LeCun, Yann A., et al. "Efficient backprop." *Neural networks: Tricks of the trade*. Springer Berlin Heidelberg, 2012. 9-48.

      Bengio, X. Glorot, Understanding the difficulty of training deep feedforward neuralnetworks, AISTATS 2010
  - Constraints on multi-layer networks with BP/BPTT
    - Computationally expensive
    - The following topics
      - BP is the optimal sol? Local minimum? Crossing plateau…
        - » Some conditions must be satisfied… => Minor Issue, see ref:

LeCun, Yann; Bengio, Yoshua; Hinton, Geoffrey (2015). "Deep learning". *Nature* **521**: 436–444.

Thank You

BPTT


x_1
x_2
$s(n_3)$
$s(n_4)$
$s(n_5)$
$s(n_6)$
$s(n_7)$
$s(n_8)$
$\hat{Y}$
$s(n_3)$
$s(n_4)$
$s(n_5)$
$s(n_6)$
$s(n_7)$
$s(n_8)$
$\hat{Y}$
x_2
$s(n_6)$
$s(n_4)$
$s(n_7)$
$s(n_8)$
$\hat{Y}$